

{EPITECH}

CODEWASH

< CLEAN YOUR CODE OR GET FIRED />



CODEWASH

Code Wash Inc.

Code has become commonplace. With the discovery and rapid rise of a new energy source that converts plastic into electricity, the unfettered use of AI is spreading chaotically. An overconsumption of code is rotting the digital environment. Bugs and "code smells" abound. Code-generating AIs feed on broken code and produce more broken code.

You're a member of **Code Wash Inc.** Code washing is a booming industry, with a bright, smelly future. Be proud to be a part of it.



Legacy is the industry norm: during your professional life you will spend more time evolving existing systems than building greenfield projects. Being able to map an unknown codebase, establish a safety net with characterization tests, refactor in small reversible steps, and deliver an enhancement is a core engineering skill. In this project, you'll be given the task to improve a project that has been entirely vibe-coded by a non-developer. Prove your worth as a human and as a dev, and make it shiny.

Your mission

After several unsuccessful school reforms, the government has finally decided to tackle the biggest problem in the education system: the overly dominant presence of students. In a sweeping revolutionary move, they have replaced learners with AI-powered robots and built the well-known AI Professor Environment.

Thanks to this, a teacher can now teach a class of 142 small robots who, once trained, will join families and share their knowledge.

Some robots, making prodigious strides in audience adaptation, have begun producing projects that blend learning with entertainment. While impressive, those initiatives aren't perfect. To reduce the risk of unproductive days in the event of a failure, the team of Code Wash Inc. (your team) is tasked with stabilizing the code and cleaning it up.



Your goal for this project is to improve the project you've been given alongside this subject.

This project has been entirely vibe-coded by a non-developper. It is impressive, but you'll notice that the code is ... ugly.

It works (it may not work outside the box but trust us, it works) but it is full of **Code Smells** : structures in the code that indicate violation of fundamental design principles and negatively impact design quality.

Your first push should be the complete project in the state it's been given to you. From now on, every push is meaningful and is a step towards greatness.

You then have to **select 4 of the features** from this missions list:

Launch the project

The project is made to work on windows, in a language you might not be familiar with, with unclear dependencies. Your first task is to launch it, and most importantly **document your process**.

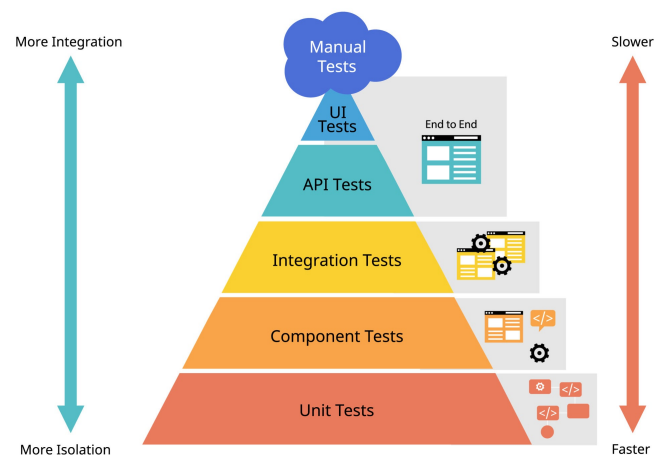
This is a good time to introduce you to **The Scout Rule**



The scout rule is **"Leave your code better than you found it."** This means every single modification you make on the project must be well-organized, supported, documented and clean. For this task, for example, it means that launching the project won't grant you any point if you haven't simplified the task for the ones who come after you. You should write a README, scripts, comment the code you come across, identify bugs, use Github in a good way... With every task in this project, realizing it is less than half the battle. You're here to clean more than to code.

Here are some general tips that might apply:

- **No refactor without tests:** first freeze current behavior with characterization tests.
- **Small, reversible steps:** one commit = one intent; keep the CI green.
- **Proof of non-regression:** Don't break public interfaces/formats without a migration plan.
- **Document** your choices (ADRs) and the remaining debt.



By the end, you will deliver a stable version of the software: existing behavior preserved, **measurable** quality (tests/coverage/CI), clarified architecture (modules, interfaces, responsibilities), and a handover package for the next team. In short: transform a fragile "black box" into a controlled, **maintainable system**. Because of this, even "Launching the game" is a valid mission that counts towards the ones you have to implement.



Following the boy scout rule is **MANDATORY** for every task you'll implement in this project

Improve performances

The main menu of the game feels quite sluggish. This is not the expected behavior for a game aimed at the dopamine-craving youth. You need to fix this.

The startup should be instant (right now our clients are wondering if they pressed the right button), with a loading screen that indicates a percentage of how much time is left before the game starts. After that, work on the game to make it snappy. Keep the animations, but make them feel better. This is closely related to user experience (UX). You'll need to measure the improvement with the adapted metrics.



Think about the scout rule for a moment. What is expected of you? Correction? Documentation? Testing? All of the above?



It's all of the above. It will always be all of the above.

Bug Reporting

There is a button on the main menu that should be about reporting a bug, but it doesn't do much at the moment. Add bug reporting as a feature.

You need to provide a way for the appropriate contact to be configured, and to send him a complete bug report via the button. Bug reporters should be able to add screenshots, game files, their error logs... How do you send this information? How do you test that it has been received?



As a rule of thumb (again, for every feature), you should provide full coverage (unit testing) for any additional code you push, and about 80% of the code you modify.

Translation

Add a way to change the language of the game and a translation in another language of your choice. This will require a new input in the main menu, preferably a button, and a list of the possible languages. Because this is a legacy project, you should not hardcode anything but instead make choices that would render trivial the implementation of the subsequent translations.

Themes

Themes have been implemented but don't work well. Not all images change, games are not impacted, sizing is incoherent... Fix those problems and add two new themes.

The current theme menu is fine, but in order to fit two new themes you'll need to add a scrolling bar. Image sizes should be standardized so that not deformation or displacement occurs when changing the theme. Games should change, including the "crafting table" game for example, where images should be about the new theme and not the old one. Music and sounds should change too.

Then, you should document how to add a new theme. This feature will be used by people who are not technical, so make sure to streamline the process as much as you can.



If you find some hardcoded values that are minecraft-themed, and you do not feel like you can change them easily with the theme, pick more standard names / branding. You'll need to justify this decision during your defense.

New game

Implement a new game about trivia, using [this API](#). It should integrate seamlessly along the existing games. Adding new games is a big feature, therefore we will be less exigent about making this task simple for those who come after, but you still should keep this goal in mind.

The UI should be coherent with the other games, reusing fonts, GUI elements...

Beware, there are a lot of systems that hinge on mini-games (archivements, levels...) The overall game should still work, even if this new mini-game is added and not linked to those systems.

New game+

This mission can only be selected if you have implemented the precedent one and added a trivia game.

You will have to full integrate the game with any system (archivements, levels). You need to add archivements, and collectables for the appropriate levels. These additions must be completly seamless, and no difference should be seen between this new game ant the original ones.

Improvements in the architecture of the project in order to simplify the process will be a part of this mission. Document everything in order to make it easier for someone who would want to add a new game in the future.

v1

{EPITECH}